Repsy Case Study: Claude Code Referansı

Konum önerisi: `docs/case-study/REFERENCE.md`. Tarih: 2026-10-23.

Bu dosya, "Repsy: depolama, ölçek ve arıza" case study'sini yaparken web araması yapmadan çalışabilmen için yazıldı. Önce bölüm 0'ı oku, sonra görev hangi aşamadaysa o bölüme git. Tamamını her seferinde yeniden okuma.

---

0. Çalışma kuralları

Token tasarrufu

- Bu dosyada cevabı olan bir şey için web araması yapma. "DOĞRULA" etiketli maddeler hariç.
- Kod tabanını okurken önce `grep -rn` / `find` ile hedefi bul, sonra yalnızca ilgili dosyaları oku. Tüm repoyu açma.
- Uzun çıktıları (Docker logları, k6 özeti, test raporu) ekrana basma. Dosyaya yönlendir (`> results/log.txt`), sonra `tail -n 40` veya `grep` ile bak.
- Her aşamanın sonunda 5-10 satırlık özet yaz: ne değişti, ne ölçüldü, neyi bilmiyoruz.

Dürüstlük

- Sayı uydurma. Rapordaki her sayı `results/` altındaki ham bir dosyadan gelmeli. Ölçemediysen "ölçülmedi" yaz.
- Bu dosyadaki kod örneklerinde geçen **endpoint, tablo ve sütun adları yer tutucudur** (`/api/packages`, `artifact`, `storage_key` gibi). Gerçek adları Repsy kodundan öğren, tahmin etme. Emin olmadığın bir şeyi "emin değilim" diye belirt.
- Repsy'de olmayan bir şeyi (ör. Redis) var gibi anlatma.
- Bir sonuç beklentiyle çelişiyorsa gizleme, olduğu gibi yaz. Negatif sonuç da case study için değerlidir.

Güvenlik

- Yıkıcı komutları (`rm -rf`, `docker volume prune`, `git reset --hard`, `DROP`) önce kullanıcıya sor.
- Şifre, anahtar, token commit'leme. Örnek değerleri `.env.example`'a koy.
- Arıza deneylerini yalnızca bu proje için kurulan Docker ortamında çalıştır. Ana diski doldurma, `/proc/sys/vm/drop_caches` gibi host ayarlarına dokunmadan önce sor.

- Her aşama ayrı commit olsun. Kullanıcı her diff'i okuyacak, anlamadığı kodu savunamaz. Karmaşık bir seçim yaptıysan commit mesajına ve ADR'ye gerekçeyi yaz.

---

1. Hedef ve teslimatlar

Repsy: REST API'li paket deposu. Spring Boot 3.5.x, Java 21, PostgreSQL, MinIO. İki depolama stratejisi var: dosya sistemi ve nesne depolama (MinIO).

Case study dört soruya cevap vermeli:

1. İki strateji aynı yükte nasıl davranıyor? (gecikme, throughput, hata oranı)
2. Bilerek arıza yaratınca ne bozuluyor? (özellikle DB kaydı ile depodaki dosya arasında tutarsızlık)
3. En ciddi tutarsızlığı nasıl düzelttik ve düzeltmenin işe yaradığını nasıl kanıtladık?
4. (İsteğe bağlı) Okuma yoluna Redis önbelleği ne kazandırdı?

Teslimatlar:

- `results/` altında ham ölçüm ve arıza çıktıları, `results/environment.md` makine bilgisi
- Tekrarlanabilir betikler (`load/`, `chaos/`), tek komutla çalışan
- Başarısız test (önce) ve düzeltme (sonra)
- Post-mortem(ler) ve ADR'ler (bölüm 11 ve 12)
- `docs/case-study/ARTICLE.md`: tek sayfa yazı, her sayının yanında hangi ham dosyadan geldiği

---

2. Önce keşfet (bilmediklerimiz)

Aşama 1'de kod okunarak cevaplanacak sorular. Cevapları `docs/case-study/00-notlar.md` dosyasına yaz:

- İki strateji hangi arayüz/sınıflarla soyutlanmış? Strateji nerede ve nasıl seçiliyor?
- Yükleme isteğinin yolu: controller, servis, depolama, veritabanı. DB yazımı ile depo yazımı aynı transaction'da mı, hangi sırayla?
- Bir adım başarısız olursa kodda ne oluyor? (hata yutuluyor mu, geri alma var mı)
- Gerçek endpoint'ler, tablo ve sütun adları, depolama anahtarının nasıl üretildiği (içerik hash'i mi, UUID mi, yol mu).
- Mevcut testler ve nasıl çalıştırılıyor.
- docker-compose dosyası var mı, hangi imajları kullanıyor.

---

3. Engelleyici sorun: MinIO imajı (önce bunu kontrol et)

Araştırma bulguları (kaynaklar bölüm 13'te):

- MinIO Community Edition deposu 25 Nisan 2026'da arşivlendi.
- `minio/minio` ve `minio/mc` imajları 11 Eylül 2026'da Docker Hub'dan kaldırıldı (bloom projesi issue #828'de saptanmış). Compose dosyası `minio/minio` çekiyorsa artık kırılmış olabilir.

Yapılacaklar:

1. `docker compose pull` ile durumu gör.
2. İmaj çekilemiyorsa bir seçenek belirle ve **digest ile sabitle**:
   - `quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z` (DOĞRULA: bazı kaynaklar Eylül 2026 sonundan beri Quay'in anonim çekmeleri de reddettiğini yazıyor)
   - Testcontainers dokümanında örnek olarak kullanılan bir fork: `pgsty/silo` (DOĞRULA)
   - Alternatif S3 uyumlu depo: Garage. Sağlık kontrolü yolu farklı (`/minio/health/live` çalışmaz).
3. Seçimi ADR-003 olarak yaz (bölüm 12). Bu karar case study'de "arıza yönetimi" hikayesinin parçası.

---

4. Tutarlılık tasarımı (case study'nin kalbi)

Sorun

DB ile nesne deposu arasında atomik işlem yoktur (dual write). İki yazma sırası da bozuk sonuç verir:

- Önce DB, sonra depo: depoda dosyası olmayan "hayalet" kayıt. Kullanıcı 404/500 görür.
- Önce depo, sonra DB: DB'de kaydı olmayan "sahipsiz" (orphan) dosya. Görünmez ama sızıntı.

Gerçek örnek: Harbor'da (CNCF registry) S3 arka ucunda depodaki blob'ların yaklaşık %87'sinin DB kaydı yoktu. Raporlayan ekibe göre 1.254.732 blob'un 1.090.780'i sahipsizdi (~74 TB). Kök neden: blob depoya yazılıyor, istemciye `201 Created` dönüyor, DB yazımı sonradan yapılıyor;

DB yazımı geçici hatayla başarısız olunca hata yutuluyor. Garbage collector yalnızca DB'deki kayıtlara baktığı için bu blob'ları hiç görmüyor. Önerilen çözüm: DB'ye önce `pending` kayıt yaz, blob'u taşı, kaydı `committed` yap, ancak ondan sonra 201 dön; eski `pending` kayıtları temizlik işi süpürsün.

Değişmezler (testlerde assert et)

- **I1:** Okuma API'si yalnızca `COMMITTED` kayıtları döner.
- **I2:** Her `COMMITTED` kaydın blob'u vardır ve boyut ile sha256 eşleşir.
- **I3:** Reconciler çalıştıktan sonra yaşı T'den büyük sahipsiz blob ve `PENDING` kayıt kalmaz.

Yükleme akışı

1. `INSERT` kayıt, `status='PENDING'`, `upload_key`, `created_at` (kendi transaction'ı, hemen commit).
2. Blob'u akışla yaz, sha256'yı yazarken hesapla (`DigestInputStream`).
    - **Dosya sistemi:** aynı dizinde geçici dosya, yaz, `FileChannel.force(true)`, `Files.move(tmp, final, ATOMIC_MOVE)`, üst dizini fsync. Geçici dosya ve hedef **aynı dosya sisteminde** olmalı, yoksa rename atomik olmaz.
    - **MinIO:** nihai anahtara `putObject`. S3 PUT nesne düzeyinde atomiktir, yarım nesne görünmez. Çok parçalı yüklemede hata olursa `abortMultipartUpload`.
3. Tek transaction'da `UPDATE status='COMMITTED', sha256, size`.
4. İstemciye `201`'i **ancak şimdi** dön.

Dizin fsync (Java, Linux'ta çalışır, bazı platformlarda `IOException` atabilir, yakala):

```
try (FileChannel dir = FileChannel.open(parent, StandardOpenOption.READ)) {

    dir.force(true);

}
```

İçerik-adresli anahtar (sha256) yeniden denemeyi güvenli yapar: aynı içerik aynı anahtara gider. Yükleme bitmeden hash bilinmediği için ilk adımda geçici bir `upload_key` kullan. (Repsy'nin mevcut anahtar şemasını önce öğren, bu öneri ona uyarlanır.)

Silme

Önce soft delete (`status='DELETED'`), sonra ayrı bir temizlik adımı blob'u silip kaydı kaldırır. Nexus Repository'nin "soft delete + compact blob store" modeli.

Reconciler (zamanlanmış iş)

`@Scheduled` + tekil kilit (ör. ShedLock) ile, **dry-run modu** olsun:

- (a) `PENDING` ve yaşı > T: blob varsa sil, kaydı sil.
- (b) Depo listesi eksi DB anahtarları, yaşı > T: sil. Bu, Harbor'ın GC'sinin kaçırdığı sahipsiz blob sınıfı.
- (c) `COMMITTED` ama blob yok: **silme**. Logla, metrik/alarm üret. İnsan kararı gerekir.
- T, en uzun makul yükleme süresinden büyük olmalı (başlangıç önerisi: 1 saat, ayarla). Aksi halde süren yüklemeyi silersin.

Outbox / saga gerekli mi?

Yükleme yolu için **hayır**: PENDING/COMMITTED ve reconciler yeterli. Tek servis, tek DB. Outbox yalnızca "blob silme" gibi yan etkiler için düşünülebilir (`cleanup_queue` tablosu, worker uygular). Bu kararı ADR-002'de gerekçelendir; "neden saga yok" cevabı mülakatta iyi sinyal.

---

5. Benzer projeler (ne öğrenilir)

Proje	**Ne öğrenilir**
goharbor/harbor #23199	Sahipsiz blob vakası, PENDING/COMMITTED önerisi, depo taramalı temizlik
distribution/distribution	GC, depodaki manifest'lerden işaretleme (mark) ve silme (sweep) yapar. "Gerçeğin kaynağı" kararı
sonatype/nexus-public	Soft delete, ayrı "compact blob store" görevi, "reconcile component database from blob store" onarım görevi
pulp/pulpcore #7806	S3'ten büyük artifact indirirken worker'ın OOM ile ölmesi. Orphan cleanup koruma süresi (`ORPHAN_PROTECTION_TIME`)
dzikoysk/reposilite	Depo başına dosya sistemi veya S3 sağlayıcısı seçimi

Proje	Ne öğrenilir
verdaccio + verdaccio-aws-s3-storage	Depolama plugin arayüzü, blob ile metadata ayrımı
go-gitea/gitea	`[storage.packages] STORAGE_TYPE = minio`, içerik-adresli dedup, cleanup job
paulcwarren/spring-content	Spring Boot'ta fs/s3/jpa depolama soyutlaması (`ContentStore`)
Shopify/toxiproxy	TCP seviyesinde arıza enjeksiyonu
testcontainers-java	MinIO ve Toxiproxy modülleri, CI'da tekrarlanabilir arıza testi
grafana/k6	Açık/kapalı yük modeli, büyük dosya yükleme notları

Yıldız sayıları ve son aktivite tarihleri bu araştırmada doğrulanmadı. README'ye eklemeden önce elle kontrol et.

---

6. MinIO notları

- Yarım multipart yüklemeleri MinIO kendisi temizler. `stale_uploads_expiry` varsayılanı 24 saat, `stale_uploads_cleanup_interval` 6 saat (2022 tarihli bakımcı yanıtı, DOĞRULA: `mc admin config get local api`). Anında temizlik için `AbortMultipartUpload`.
- `AbortIncompleteMultipartUpload` lifecycle kuralını MinIO kabul eder ama saklamaz (issue #19115). AWS S3'te çalışan kural MinIO'da sessizce yok sayılır. "Taşınabilirlik varsayımı" bulgusu olarak yazılabilir.
- Yarım yüklemeleri listele ve temizle:

```
mc ls local/repsy --incomplete --recursive

mc rm local/repsy --incomplete --recursive --older-than 7d --force
```

- Sahipsiz ve hayalet sayımı:

```
mc ls --recursive --json local/repsy | jq -r '.key' | sort > /tmp/store.txt
```

psql -At -c "select storage_key from artifact where status='COMMITTED'" | sort > /tmp/db.txt

comm -23 /tmp/store.txt /tmp/db.txt   # depoda var, DB'de yok (sahipsiz)

comm -13 /tmp/store.txt /tmp/db.txt   # DB'de var, depoda yok (hayalet)

(Tablo ve sütun adları yer tutucudur. `mc` takma adı `local` ve bucket adı `repsy` da öyle.)

---

7. Spring Boot ayarları ve tuzaklar

spring.servlet.multipart.max-file-size=60MB   # varsayılan 1MB

spring.servlet.multipart.max-request-size=61MB   # varsayılan 10MB

spring.servlet.multipart.file-size-threshold=0   # varsayılan zaten 0: dosya hemen diske yazılır

server.tomcat.max-swallow-size=64MB

Deney (önce/sonra ölç, etkisi bu araştırmada ölçülmedi):

spring.threads.virtual.enabled=true

- `MultipartFile.getBytes()` kullanma, `getInputStream()` ile akıt.
- Multipart yüklemede dosya önce Tomcat'in geçici dizinine, sonra hedefe yazılır: **çift yazım**. Dosya sistemi stratejisinin gecikmesine bu eklenir. Benchmark'ta belirt veya ham gövdeli bir `PUT` endpoint'i (`application/octet-stream`) ekleyip multipart ayrıştırmayı atla.
- İndirmede `StreamingResponseBody` veya `InputStreamResource` kullan, MinIO yanıt akışını doğrudan yanıta kopyala, `Content-Length` ver. Pulp #7806 dersi: S3 nesnesini belleğe tamamen almak büyük dosyada OOM üretir. 50 MB indirme testinde heap'i izle.
- `MaxUploadSizeExceededException` düzgün bir 413'e çevrilmeli.

---

8. k6 yük testi

Fixture'lar (rastgele içerik sıkıştırılamaz):

mkdir -p load/fixtures

```
head -c 1024      /dev/urandom > load/fixtures/1kb.bin

head -c 1048576   /dev/urandom > load/fixtures/1mb.bin

head -c 52428800  /dev/urandom > load/fixtures/50mb.bin
```

`load/upload.js` (endpoint ve alan adları yer tutucudur, gerçek API'ye uyarla):

```
import http from 'k6/http';

import { check } from 'k6';

const SIZE = __ENV.SIZE || '1mb';

const BASE = __ENV.BASE_URL || 'http://localhost:8080';

const bin = open(`./fixtures/${SIZE}.bin`, 'b');   // her VU'da kopya: 50mb'de VU sayısını düşük
tut

export const options = {

  discardResponseBodies: true,

  summaryTrendStats: ['avg', 'med', 'p(90)', 'p(95)', 'p(99)', 'max'],

  scenarios: {

    warmup: {

      executor: 'constant-arrival-rate', rate: Number(__ENV.RATE || 10), timeUnit: '1s',

      duration: '1m', preAllocatedVUs: 20, maxVUs: 100, tags: { phase: 'warmup' },

    },

    measure: {

      executor: 'constant-arrival-rate', rate: Number(__ENV.RATE || 10), timeUnit: '1s',

      duration: '5m', startTime: '1m', preAllocatedVUs: 20, maxVUs: 100, tags: { phase: 'measure'
},

    },
```

```
  },
  thresholds: {
    'http_req_failed{phase:measure}': ['rate<0.01'],
    'dropped_iterations': ['count==0'],
  },
};
export default function () {
  const name = `pkg-${__VU}-${__ITER}-${Date.now()}`;
  const res = http.post(`${BASE}/api/packages`,
    { file: http.file(bin, `${name}.bin`), name },
    { tags: { op: 'upload', size: SIZE } });
  check(res, { 'status 201': (r) => r.status === 201 });
}
```

Başlangıç hızları (ayarla): 1 KB için `RATE=100`, 1 MB için `RATE=20`, 50 MB için `RATE=1-2`.

Çalıştırma:

k6 run -e SIZE=1mb -e RATE=20 --summary-export=results/raw/fs-upload-1mb-run1.json load/upload.js

İndirme ve listeleme betikleri aynı iskeleti kullanır: `setup()` içinde N paket yükle, sonra GET'leri ölç.

Her koşu için raporla: p50, p95, p99, `http_reqs/s`, MB/s (hız × boyut), `http_req_failed`, `dropped_iterations`, uygulama konteynerinin CPU/RAM'i (`docker stats --no-stream`), k6 sürecinin CPU'su.

---

9. Ölçüm tuzakları (kaçın)

1. `constant-vus` gibi kapalı modelle p99 raporlama. Sistem yavaşlayınca gelen yük de azalır, p99 iyimser çıkar (coordinated omission). `constant-arrival-rate` kullan.
2. `dropped_iterations > 0` ise koşu geçersiz sayılır (VU havuzu yetmedi). `maxVUs`'u artır veya hızı düşür, ya da rapora açıkça yaz.
3. Isınmasız ölçüm. İlk dakika JIT ve bağlantı havuzunu ölçer, warmup senaryosunu hariç tut.
4. Dosya boyutlarını karıştırma. Her boyut ayrı koşu, ayrı satır.
5. k6 ve uygulama aynı CPU'da. Uygulamaya `cpus` ve `mem_limit` ver, k6'yı ayrı çekirdekte ya da makinede çalıştır.
6. Page cache. Dosya sistemi indirmeleri RAM'den gelir. **Sıcak** (aynı dosyalar tekrar) ve **soğuk** (önbellek boşaltılmış ya da çalışma seti RAM'den büyük) koşuyu ayrı raporla. Linux host'ta soğuk koşu: `sync; echo 3 | sudo tee /proc/sys/vm/drop_caches` (önce kullanıcıya sor). Docker Desktop/macOS'ta önbellek VM içindedir, bunu sınırlama olarak yaz.
7. 50 MB x çok VU, k6'nın kendi belleğini patlatır (`open()` VU başına kopya, multipart ek kopya, k6 #2311). 50 MB senaryosunda hızı düşük tut, k6 sürecini izle.
8. 1 KB testi depolamayı değil HTTP/framework yükünü ölçer. Öyle yorumla.
9. Multipart geçici dosya çift yazımı dosya sistemi stratejisini cezalandırır, belirt.
10. Tek koşu yeterli değil: en az 3 tekrar, medyan ve aralığı ver. Makine, işletim sistemi, Docker sürümü ve kaynak limitlerini `results/environment.md`'ye yaz.
11. Yerel makine üretim değildir: gerçek ağ yok, disk paylaşımlı. Sonuçları "bu makinede göreli karşılaştırma" olarak sun.

---

10. Arıza enjeksiyonu

Toxiproxy ile ağ arızası

Compose eklentisi (imaj etiketi DOĞRULA):

```
services:
  toxiproxy:
    image: ghcr.io/shopify/toxiproxy:2.12.0
    ports: ["8474:8474", "15432:15432", "19000:19000"]
```

```
  app:
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://toxiproxy:15432/repsy
      MINIO_ENDPOINT: http://toxiproxy:19000
    deploy:
      resources:
        limits: { cpus: "2", memory: 1g }
```

(Ortam değişkeni adlarını Repsy'nin gerçek yapılandırmasına göre düzelt.)

Proxy'leri oluştur:

```
curl -s -XPOST localhost:8474/proxies -d
'{"name":"pg","listen":"0.0.0.0:15432","upstream":"postgres:5432"}'

curl -s -XPOST localhost:8474/proxies -d
'{"name":"minio","listen":"0.0.0.0:19000","upstream":"minio:9000"}'
```

Toxic'ler:

```
# gecikme

curl -s -XPOST localhost:8474/proxies/pg/toxics -d
'{"name":"lat","type":"latency","stream":"downstream","attributes":{"latency":1000,"jitter":100}}'

# asılı bağlantı (timeout=0: bağlantıyı kapatmadan veriyi bekletir, read timeout eksikliğini ortaya çıkarır)

curl -s -XPOST localhost:8474/proxies/minio/toxics -d
'{"name":"hang","type":"timeout","attributes":{"timeout":0}}'

# yükleme ortasında kopma: 5 MB sonra kes

curl -s -XPOST localhost:8474/proxies/minio/toxics -d
'{"name":"cut","type":"limit_data","stream":"upstream","attributes":{"bytes":5242880}}'

# RST
```

```
curl -s -XPOST localhost:8474/proxies/pg/toxics -d
'{"name":"rst","type":"reset_peer","attributes":{"timeout":500}}'

# proxy'yi kapat / aç, toxic'i sil

curl -s -XPOST localhost:8474/proxies/pg -d '{"enabled":false}'

curl -s -XDELETE localhost:8474/proxies/pg/toxics/lat
```

Toxic'ler yönlüdür, varsayılan `downstream`.

Süreç arızası (yükleme sürerken)

```
curl -s -o /dev/null -w '%{http_code}\n' -F file=@load/fixtures/50mb.bin -F name=x
localhost:8080/api/packages &

sleep 1; docker compose kill -s SIGKILL minio      # ya da: postgres

wait; docker compose start minio

docker compose pause postgres     # donma simülasyonu; sonra: docker compose unpause
postgres
```

Disk dolması (dosya sistemi stratejisi)

Küçük bir tmpfs volume, ana diske dokunmaz:

```
volumes:

  pkgdata:

    driver: local

    driver_opts: { type: tmpfs, device: tmpfs, o: "size=60m" }

# app: volumes: ["pkgdata:/data/packages"]   -> 50 MB'yi iki kez yükle, ENOSPC beklenir
```

- tmpfs konteynerin bellek limitine sayılır. `mem_limit`'i tmpfs boyutu + uygulama için yeterli ver, yoksa OOM olur.
- Daha gerçekçisi (root gerekir, önce sor): `fallocate -l 100M disk.img && mkfs.ext4 disk.img && sudo mount -o loop disk.img /mnt/repsy-small`, sonra bind mount.
- MinIO'nun küçük diskteki davranışı doğrulanmadı, gözlemle ve yaz.

Her deneyden sonra

Bölüm 6'daki sahipsiz/hayalet sayımını çalıştır, I1-I3 değişmezlerini kontrol et, sonucu tabloya yaz: {senaryo x strateji} için istemci ne gördü (HTTP kodu), DB'de yarım kayıt var mı, depoda sahipsiz dosya var mı.

Testcontainers iskeleti (önce başarısız test)

```
@Testcontainers

class UploadConsistencyIT {

  static Network net = Network.newNetwork();

  @Container static PostgreSQLContainer<?> pg = new
PostgreSQLContainer<>("postgres:16-alpine")

      .withNetwork(net).withNetworkAliases("pg");

  @Container static MinIOContainer minio = new MinIOContainer("<pinli minio imajı>")

      .withNetwork(net).withNetworkAliases("minio");

  @Container static ToxiproxyContainer toxi = new
ToxiproxyContainer("ghcr.io/shopify/toxiproxy:2.12.0")

      .withNetwork(net);

  @Test void dbOutageAfterBlobWrite_leavesNoOrphanAfterReconcile() {

    // 1) blob yazıldıktan sonra pg proxy'yi kapat (hook ya da spy ile zamanla)

    // 2) upload çağır, 5xx beklenir

    // 3) proxy'yi aç, reconciler'ı T süresi ilerletilmiş saatle bir kez çalıştır

    // 4) assertNoOrphans(): depo anahtarları, COMMITTED anahtarların alt kümesi; PENDING
kayıt yok

  }

}
```

MinIO imaj adı, Toxiproxy kontrol API'si ve port aralığı modül sürümüne göre değişebilir (DOĞRULA). İlk test, düzeltme yapılmadan **kırmızı** olmalı. Kırmızı çıktıyı `results/failures/` altına kaydet, sonra düzelt.

---

11. Post-mortem şablonu (blameless)

Google SRE Book bölüm 15 ve ek D'den uyarlandı. Her arıza deneyinin bulgusu bir sayfa:

PM-00X: <başlık>

Tarih / Yazar / Durum

Özet (2-3 cümle)

Etki (sayılarla: başarısız istek, sahipsiz nesne sayısı/MB, süre)

Tetikleyici ve katkıda bulunan faktörler (kişi değil, sistem)

Zaman çizelgesi (t+0s ...)

Tespit: nasıl fark edildi, edilemedi mi

Ne iyi gitti / ne kötü gitti / nerede şanslıydık

Aksiyonlar (sahip, öncelik, test/commit bağlantısı)

Kural: "insan hatası" sonuç değil başlangıç noktasıdır. Suçlayıcı dil kullanma.

---

12. ADR listesi

Her biri kısa: bağlam, karar, alternatifler, sonuçlar.

- **ADR-001** Depolama strateji arayüzü (`StorageBackend`: put/get/delete/list/exists) ve iki implementasyonun aynı sözleşme testlerini geçmesi
- **ADR-002** Yazma sırası: PENDING, blob, COMMITTED, 201 commit'ten sonra. Neden saga/outbox yok
- **ADR-003** Nesne deposu tedarik riski: MinIO arşivlendi, imaj sabitleme ve alternatif
- **ADR-004** Reconciler: zaman penceresi T, dry-run, COMMITTED-blobsuz kayıtları otomatik silmeme

- **ADR-005** Redis yalnızca metadata/listeleme için, cache-aside + TTL, Redis düşerse DB'ye düş (fail-open). 50 MB blob'ları Redis'e koyma
- **ADR-006** Ölçüm metodolojisi: açık model, sıcak/soğuk ayrımı, kaynak limitleri, geçersiz koşu kuralı

---

13. Aşama aşama kontrol listesi

Her aşama bir commit. "Bitti" ölçütü aşamanın sonunda yazılı.

- ☐ **0. Ortam ve MinIO imajı.** Bölüm 3. Bitti: `docker compose up` yeşil, imajlar pinli, kaynak limitleri var.
- ☐ **1. Keşif (salt okunur).** Bölüm 2. Bitti: `00-notlar.md` yazıldı, gerçek endpoint/tablo adları ve yazma sırası belgelendi.
- ☐ **2. Strateji arayüzü ve sözleşme testleri.** Bitti: aynı test paketi iki implementasyonda da geçiyor.
- ☐ **3. Başlangıç ölçümü.** Bölüm 8-9. Bitti: strateji x boyut x işlem için 3 tekrar, `dropped_iterations=0`, sıcak/soğuk ayrı, ham dosyalar `results/raw/`, tablo betikle üretildi.
- ☐ **4. Arıza matrisi.** Bölüm 10. Bitti: {MinIO kill, PG kill, PG pause, disk dolu, latency, limit_data} x {dosya sistemi, MinIO} için HTTP sonucu + sahipsiz/hayalet sayısı tabloda.
- ☐ **5. Başarısız test.** En ciddi tutarsızlık için kırmızı test commit edildi.
- ☐ **6. Düzeltme.** Bölüm 4. Bitti: PENDING/COMMITTED + reconciler, test yeşil, sahipsiz sayısı 0, arıza betiği düzeltmeden önce ve sonra çalıştırıldı.
- ☐ **7. Post-mortem(ler).** Bölüm 11.
- ☐ **8. (İsteğe bağlı) Redis.** Bitti: listeleme p95 önce/sonra, hit oranı, Redis kesintisi deneyi.
- ☐ **9. ADR-001..006.**
- ☐ **10. Yazı.** `ARTICLE.md`: Bağlam, Problem, Kararlar ve gerekçeleri, Ölçülen sonuç (tablo), Arıza ve post-mortem, Farklı yapsaydım, Sınırlamalar. Her sayının yanında kaynak dosya.

---

14. Kaynaklar ve doğrulama durumu

Kaynaklar (araştırmada okundu veya aratıldı):

- Harbor #23199: https://github.com/goharbor/harbor/issues/23199
- Pulp #7806: https://github.com/pulp/pulpcore/issues/7806
- MinIO #19115: https://github.com/minio/minio/issues/19115
- k6 #2311: https://github.com/grafana/k6/issues/2311

- Toxiproxy: https://github.com/Shopify/toxiproxy
- Testcontainers MinIO modülü: https://java.testcontainers.org/modules/minio/
- Testcontainers Toxiproxy modülü: https://java.testcontainers.org/modules/toxiproxy/
- Docker tmpfs: https://docs.docker.com/engine/storage/tmpfs/
- Google SRE postmortem kültürü: https://sre.google/sre-book/postmortem-culture/
- Google SRE örnek post-mortem: https://sre.google/sre-book/example-postmortem/

Doğrulanmamış, kullanmadan önce kontrol et

- MinIO imaj kaldırma tarihi ve Quay'in anonim çekmeleri reddedip reddetmediği
- `stale_uploads_expiry` ve temizlik aralığı varsayılanları (2022 tarihli yanıt)
- Toxiproxy ve MinIO imaj etiketleri, Testcontainers modül API'si
- Virtual thread ve Redis'in Repsy üzerindeki etkisi (ölçülmedi, deney olarak öner)
- Yıldız sayıları ve proje son aktivite tarihleri
- Midas/Iyzico rol gereksinimleri (case study'nin hedef alıcıları, bu dosyada kullanılmadı)

Yanıltıcı olabilecek her şey için kural: bir şey bu dosyada "DOĞRULA" diyorsa yerelde bir komutla doğrula, sonucu notlara yaz, sonra kullan.